

Flaky-free Python & Pytest Check List v 1.0 (19 Oct 2025)

Преамбула

Key targets

Чек лист направлен на предотвращение Python & Pytest related нестабильности (flaky) на всех этапах разработки.

Чек лист не направлен на получение исчерпывающего материала по всем возможным причинам и проявлениям flaky.

Цель: систематизировать и разобрать максимальное кол-во и вариации Python & Pytest related flaky.

Дополнительные материалы, связанные с flaky, даны в ссылках в конце.

Целевая реализация: cPython3.

Порог вхождения

Ожидается, что читатель знаком с базовыми понятиями Python 3 (примитивные типы, коллекции, классы, принцип работы памяти) и pytest (тест, фикстура, коллектинг), а также ключевыми концепциями тестирования (требования, пирамида и типы тестов, техники тестирования)

Структура

Чек лист построен +- в соответствии с жизненным циклом авто теста глазами разработчика - автотестера:

- Проектирование ("думаю о тесте у себя в голове")
- Реализация (ака написание) ("пишу локально") -- самый большой и технически важный раздел. Разбит на 2 части: "изолированные" и "протекающие" flaky.
- Ревью (для авторов кода и ревьюеров) -- чек лист не стремится дать исчерпывающий материал по всем практикам code review, а фокусируется на mind set и конкретных практиках "как сделать лучше и безопаснее".
- Запуск кода в CI/CD и его сопровождение.

Основная часть

Проектирование автотеста

На данном этапе тест "в голове"

- Определиться с уровнем теста в пирамиде тестирования.
General rule of thumb: Если тест можно сделать более низкоуровневым (компонент вместо интеграционного) -- это хорошая идея.
Избегай соблазна покрывать все end-to-end UI тестами (верх пирамиды) -- они хрупкие.
Пример: happy path пользователя на сайте.
Вместо одного браузерного теста более стабильным вариантом может стать

- отдельно e2e API тест и отдельно UI тест с замканными ответами с бэка.
- Больше внешних интеграций и зависимостей == больше точек отказа.
По возможности мокируй зависимости и пиши тесты, сфокусированные на тестировании одной интеграции, другие мокируй.
Еще лучше если есть возможность мокнуть все зависимости и сфокусироваться на тестировании бизнес логики нашего продукта.
- Небольшой сфокусированный тест, как правило, стабильнее чем огромный тест на "все подряд".
Поддержка и дебаг нескольких небольших тестов проще чем одного большого.
Границы "большого" определяются эмпирически.
Пример: есть большой тест на регистрацию пользователя в системе, проведение платежа, проверку документов.
Три секции отказа: регистрация, платеж, документы.
Этот тест распили на 2: 1) регистрация; 2) получение уже зареганного юзера и проведение им платежа + проверка документов -- регистрация вынесена за скоуп теста.

Реализация, часть I, "изолированное" flaky

Тут собраны кейсы, способные сломать один тест, они не "протекают" на соседние тесты и всю систему.

Floating-point arithmetic

Кейсы потери точности и/или накопления ошибки в связи с особенностями реализации плавающей арифметики.

- Произвольные вычисления с дробными числами, отличающимися на несколько порядков, приведут к потере точности.
Пример:

```
n = 100_000_000
for i in range(4):
    n *= 0.001 # OK, 0.0001

n = 100_000_000
for i in range(5):
    n *= 0.001 # Error, 0.000000100000000000000000087...
```

Использовать встроенные точные типы Decimal, Fraction или библиотеки для научных вычислений (scipy, sympy) или интеграции с computer algebra systems (Wolfram Mathematica)

Unordered comparison

- Потеря порядка при переходе упорядоченные -> неупорядоченные множества и обратно.
См пример test_loss_of_order.

Решение: если порядок важен, то его нужно явно создать или воссоздать.
- Порядок там где он не важен. Если нужно сравнить две коллекции без учета порядка, то достаточно обернуть их решение по типу collections.Counter.

Время: часы и таймеры

- При замерах времени нужно понимать какой вопрос: "который час" или "сколько прошло?"

 - Который час: `time.time`, `datetime.datetime` -- настенные часы
 - Сколько прошло: `time.monotonic`, `time.perf*` -- секундомер

Неправильный выбор приведет к flaky поведению, см пример 01. Замеряем время из презентации.

- Для замеров продолжительности событий (bench marking), длящихся порядка миллисекунд и меньше стоит выбирать `time.perf_counter*`-подобные варианты вместо `time.time` из-за более высокого разрешения первого.
- Не всё можно помереть: замер конкретных и параллельных задач не всегда доступен "из коробки".
При реализации такого решения нужно убедиться, что оно работает точно. замерив несколько параллельных задач с разной заранее известной продолжительностью и верифицировать результат
- Общие правила по работе с датой и временем:
 - Timezone aware лучше чем Timezone naive.
 - UTC проще в работе чем произвольные часовые пояса из-за отсутствия DST и исторических изменений.
 - ISO строки даты-времени лучше, чем произвольные, т.к. ISO несет информацию о time zone.
 - В межкомпонентном взаимодействии лучше использовать формат даты времени с явным указанием time zone / offset, даже если есть договоренность между разработчиками что время в таком-то поясе.

см пример `test_look_at_the_time.py`.

Динамическая типизация

- Код с type annotations меньше склонен к flaky чем код, где IDE/линтер угадывает типы.
- Интеграция аннотаций в код может казаться сложной или ненужной задачей, но в long run это выгодно.
- Не нужно переписывать весь проект на аннотации + комалаенс со статик чекером за один раз:

Начни с малых шагов в любом удобном месте или месте, которое ты активно пилишь сейчас.
- Без аннотаций есть риск работы с неверным типом, что может привести к произвольно сложным проблемам

см пример `test_bad_typing.py`
- Используй линтеры и static type checkers

 - <https://github.com/typeddjango/awesome-python-typing>
 - <https://github.com/vintasoftware/python-linters-and-code-analysis>

Не злоупотребляй словарями

- ✓ простые словари
- ✓ перекладывание словаря из ручки в ручку или между компонентами

приложения

- **x** хранить сложные вложенные типы в словарях
- **xx** постоянное расползание логики парсинга и валидации словаря по телу теста.

Это сложно поддерживать и читать.

Если нужно валидировать и обрабатывать словарь, используй правильные инструменты: валидаторы, классы, модели.

(typed dict, dataclass, Pydantic).

N.B. Возможны кейсы, где упомянутые решения не подходят из-за RAM/CPU overhead.

Choose wisely.

См пример `test_dont_abuse_the_dict.py`

Рандом

Два основных желаемых свойства рандома

- Воспроизводимость. У нас должна быть возможность снова сгенерить те же самые данные
- Уникальность. Два теста (даже запущенных одновременно) не должны случайно начать генерировать одинаковый рандом.

Как получить:

- Репортировать используемый seed рандома в отчет
- Иметь возможность задать seed (например, через опциональную переменную среды)
- Не использовать кастомные слабые генераторы seed/random если можно использовать более стабильные и надежные решения "из коробки"

См пример `test_random.py`

Реализация, часть II, "протекающее" flaky и архитектура

Самое главное архитектурное правило хорошо спроектированного теста -- не трогать другие тесты и их ресурсы и не зависеть от них.

Мутабельные объекты с общим доступом

- Мутабельные объекты, передаваемые по одной ссылке в несколько тестов, должны быть read only.
- Если read only обеспечить нельзя, то обеспечить передачу разных экземпляров в каждый тест
- Глобальные состояния должны иметь read only доступ или single changer. см пример 03. Память в Python из презентации.

Утечки памяти в Python

- Ссылки на неиспользуемые объекты внутри других объектов. Следи за ссылками, особенно циклическими.
- Memory Leak via Closure Holding Reference.

```
import sys
global_data_store = []
def lambda_handler(event, context):
    global_data_store.append("some_data_to_leak" * 1024) # Add a relatively
large string
    print(f"Current memory usage: {sys.getsizeof(global_data_store)} bytes")
    return {
        'statusCode': 200, 'body': f'Data added. Global store size:
{len(global_data_store)} elements.'
    }
```

Диагностика

- TraceMalloc
- ObjGraph
- memory_profiler
- pytest-leaks
- Смотрим глазами на htop/дашборд с памятью

Утечки системных ресурсов

Подразумеваем утечки не памяти внутри Python процесса, а ресурсов ОС или других приложений:

файловые дескрипторы, сокеты, подключения к БД, запущенные процессы.

- Обязательно явно контролировать закрытие ресурса, особенно в случае исключений.
Три основных способа:
 - покрытие try/except/finally
 - интеграция context manager with ... as: с помощью __enter__ + __exit__ или contextlib
 - setup & teardown внутри фикстуры. Возможна интеграция с вариантами выше.
см пример 04. Неуправляемые ресурсы в презентации.

Диагностика -- как и выше.

Использование фикстур

- Если жизненный цикл ресурсов не управляется явно в тесте (try/except/finally или context manager), тест должен использовать фикстуры для управления ресурсами.
- Если фикстура создает ресурс, то она же и зачищает его.
- Если ресурс создается в тесте, допустимо зачищать его в tear down фикстуры
- Часто оптимальным решением является сэтап и тирдаун внутри фикстуры.
- Фикстура не должна быть пухлой и управлять несколькими типами ресурсов сразу, иначе падение в setup или teardown может оставить dangling resources.

Маленькие фикстуры проще поддерживать и легче переиспользовать как dependency injection.

см пример 05. Неатомарные фикстуры в презентации.

Плохая изоляция

- General rule of thumb: хорошо изолированный тест не трогает другие тесты и их ресурсы и не ждет ничего от них.
- Хорошо изолированный тест не зависит от тест сета, в котором он запускается, порядка выполнения тестов в сете, и успешного прохождения других тестов в сете.

- Хорошо изолированный тест не трогает чужие ресурсы. Если есть общие ресурсы, то они управляются так, чтоб не порушить соседей.
- Три основных принципа построения хорошо изолированного теста.
 - Автаркия: тест сам сетапит свои ресурсы и сам убирает за собой (явно или через фикстуру).
 - Тест не надеется на наличие и порядок других тестов в тест сете, должна быть возможность запустить этот тест отдельно или в произвольном тест сете.
 - Тест не надеется, что другой тест (исполняемый перед ним или параллельно с ним) создаст ресурс.
 - Тест не надеется, что последующий тест уберет мусор и/или восстановит состояние системы, если эти изменения были выполнены этим тестом.
 - N.B. Последнее не значит, что тест не должен восстанавливать состояние системы или менять его на нужное себе -- это нормальный паттерн.
 - Пессимизм: тест не надеется, что перед ним есть другие тесты и что они успешно отработали и создали ресурсы.
 - Watch your step: у тестов или нет общих ресурсов, или тест не шатает им трубу.
 - Perfect case scenario: у теста полностью свои ресурсы (файлы, базы данных, процессы, память, etc). Это может быть дорого или недостижимо в принципе (состояние десктоп приложения).
 - Meh case scenario: все таки есть общие ресурсы.
 Что делать?
 - Это или read-only ресурсы
 - Или доступ к ним синхронизирован и безопасен. Например, есть прослойка, Single Responsibility которой является безопасный доступ и изменение.

Диагностика

- По возможности применять
 - или случайный порядок запуска тестов (pytest-randomly, pytest-random-order)
 - или параллельный (pytest-xdist)
 - или комбинировать

- N.B. Случайный порядок должен быть возможен в абсолютном большинстве тестов. Параллельность возможна не всегда (UI тест десктоп приложения)

Экосистемные особенности

Набор скорее анекдотических кейсов, связанных с неявными ломающими изменениями

- Переход с libC на musl приводил к падению [производительности](https://news.ycombinator.com/item?id=27379197) (<https://news.ycombinator.com/item?id=27379197>) и таймаутам тестов
- В 3.8 требование о сохранении порядка ключей словаря стало обязательным, до этого оно было особенностью имплементации cPython. Этот же код, запущенный на 3.7 другой реализации может флаковать.
- Между [3.2](https://stackoverflow.com/questions/49896202) (<https://stackoverflow.com/questions/49896202>) и [3.3](https://stackoverflow.com/questions/26115765) (<https://stackoverflow.com/questions/26115765>) поменялся способ перехода из set в упорядоченные коллекции.


```
s1 = {'a', 'b', 'c'}
s2 = {'c', 'a', 'b'}
assert tuple(s1) == tuple(s2) # True for 3.2, False for >= 3.3
```

- в pytest 8.4 обновился API `pytest.mark` что могло приводить к тихому исчезновению пометок с теста
- `pytest.xdist` нужны параметры в виде `ordered collection`, иначе могут возникнуть неожиданные коллизии

Ревью

Появится после выступления, смотрите онлайн и участвуйте в обсуждении ☺

Тест в проде

Появится после выступления, смотрите онлайн и участвуйте в обсуждении ☺

Что еще почитать?

- Системный подход к стабилизации тестов для мобильных приложений. Надежда Дегтярева, Дмитрий Макаренко. Heisenbug 2021. heisenbug.ru/archive/2021%20Moscow/talks/qopanqphk8c8y4zocwwpo
- Flaky tests / Flaky tests. Порядок имеет значение / Flaky tests. Метод. Андрей Солнцев. Heisenbug 2017 / Heisenbug 2020 / Heisenbug 2021. heisenbug.ru/archive/2020%20Moscow/persons/5vykhdaynv0cnkc9lzuqvh/
- 7 смертных грехов тестирования. Вадим Никитенко. Heisenbug 2024. heisenbug.ru/archive/2024%20Autumn/talks/7507c6f5d2b741c9bfe3d60c9bd6ba14
- Ошибки при внедрении автоматизированного тестирования и способы их предотвращения. Дмитрий Шахов. Innopolis Meetup: QA 2024. youtube.com/watch?v=RCbKPfkkdME&t=5122s
- Вывести типы из Python: проблемы анализа Python-кода

Екатерина Точилина, Вячеслав Тамарин. PiterPy 2024

youtube.com/watch?v=oc19hfdhk0I

- An Empirical Study of Flaky Tests in Python
Martin Gruber, Stephan Lukasczyk, Florian Kroiß, Gordon Fraser.
arxiv.org/pdf/2101.09077
- <https://dev.to/gewenyu99/how-to-detect-and-fix-flaky-tests-in-pytest-1k61>
- https://www.marwansarieddine.com/posts/flaky_tests
- <https://semaphore.io/blog/flaky-test-retry>
- <https://semaphore.io/blog/2017/04/20/flaky-tests.html>
- <https://www.testrail.com/blog/flaky-tests/>
- <https://pypi.org/project/flaky-tests-detection/>
- <https://pypi.org/project/pytest-flakefinder/>